(1) Differences between server and client side

1. Core idea

Where is the experiment logic implemented?

- **server-side:** logic runs on the backend (server). All data processing and variation assignment happen before content reaches the user.
- **client-side:** logic runs in the browser (frontend). Page is delivered, and Javascript modifies it dynamically to apply variations.

(2) Key Difference

Aspect	Server-side	Client-side
Experiment logic	Runs on the server	Runs in browser using Javascript.
Speed	Faster page loads	Might be slower due to DOM manipulation.
Security	More secure (logic hidden)	Less secure (code visible to user)
Measurement	More reliable (less user interruption)	Can be noisy or inconsistent.
Exposure	User assigned before page load.	User assigned after page load.
Randomization point	Server assigns user to variant early	Client assigns user later (after initial content)
Data Consistency	Easier to manage (centralized system)	More prone to missing events (JS fails to load)

Flexibility	Harder to iterate visually (developer-dependent)	Easy to change & visuals post (great for QE testing).
Cross-device consistency	Easier to maintain	Harder, especially with anonymous users.

(3) Real-world Analogy: "Chef vs waiter".

Imagine you go to a restaurant.

- Server-side (chef-prepared dish):
  - ↳ You place the order (request goes to server)
  - ↳ The chef cooks and plates everything before it arrives.
  - ↳ You receive a finished product, consistent and controlled.
- Client-side (waiter-prepared).
  - ↳ You get a basic plate with raw ingredients.
  - ↳ The waiter at your table arranges or modifies your dish.
  - ↳ Results vary based on environment (table space, waiter speed).

This shows:

- Server-side = clean, controlled, consistent
- Client-side = flexible, but prone to errors, variations, and delays.

4. Math Concept Behind it: Randomization Timing & Bias.

- Server-side randomization = before any user interaction (less biased)
- Client-side randomization = after some content is load → introduces ~~selec~~ selection bias (e.g. some users drop off before Javascript runs).

Let is
- 100 users visit your site.
- Your goal is to randomly assign users to either
  - ↳ control
  - ↳ Treatment.
- This is known as Randomization.

☑ Server-side Randomization (Math is clean).

Step-by-step
- Each user has equal probability $P = 0.5$ of being in Control or Treatment.
- No one is excluded
- You are sampling from a simple random sample (SRS) of the population.

Math Behind
Let:
- $X_i = 1$ if user i convert (i.e. success).
- $A_i \in \{0, 1\}$: assignment (0 = control, 1 = treatment)
- Your estimate of treatment effect is:

$$\hat{T}_{server} = \Sigma [X_i | A_i = 1] - \Sigma [X_i | A_i = 0]$$

Since everyone is randomized before page load:

- The assignment is independent of user behavior, devices, or browser
- This makes the estimator unbiased.

| Conclusion: Estimate is a good approximation of true treatment effect.

Client-side Randomization (Bias Risk)

Now imagine the same 100 users, but...

⚠ Some Problems

- 10% have JavaScript disabled or fail to load script.
- 5% bounce before script runs.
- Only 85 users actually get randomized.

Let's break this down mathematically

- $S_i = 1$ if user i stays long enough and JS works (i.e. eligible for randomization)
- $A_i \in \{0, 1\}$ as before (assigned group)
- X_i = observed conversion.
- Then:

$$\hat{T}_{client} = \sum [X_i | A_i = 1, S_i = 1] - E[X_i | A_i = 0, S_i = 1]$$

But not now note: This is conditional on $S_i = 1$

So you are estimating

$$\hat{T}_{client} = T \mid \text{user didn't bounce and JS succeeded.}$$

↳ This excludes users who had bad network, low patience, mobile issue → leads to selection bias.

Real-world Example with Numbers

Let's simulate 100 users coming in

↳ 20 of them are impatient or JS fails (e.g., lower-engagement users).

↳ These 20 would not get included in your experiment if it's client-side.

↳ But what if these 20 users also had a much lower conversion rate, say 2%?

Then your observed conversion rate is overestimated because:

- You're only looking at the "engaged" group who stay longer (and usually convert more)

This introduces

- Upward bias in estimating both groups' conversions
- Over-estimation of treatment effect.

Link to Math Concept: Conditioned Expectation.

↳ When conditioning on $S = 1$, we are selecting a non-random subset.

This violates the assumption that assignment is independent of outcome potential:

$$A_i \perp Y_i(1), Y_i(0)$$

In Potential outcomes framework (Rubin Causal Model), this makes estimates biased, because the assignment depends on post-treatment variables (e.g. whether JS worked)

Summary.

Concept	Server - side	Client - side.
Sample size	All users	Subset of users.
Randomization	Unconditional	Conditional on triggering JS work.
Bias	Low	High.
Mathematically	$E[Y/A]$	$E[Y/A, S=1]$
Assumption Violation	None	Yes, selection bias.
Best Use case	Backend logic / reliability	UI changes, visuals fast only.

(2) Implications for Experiments.

When you run experiments on the client-side (browser via Javascript), it creates new risk and challenges than compromise:

- Randomization
- Measurement Accuracy:
- Interpretation of result.

Core Implications:

1. Exposure Bias

What it means: Not every user gets a fair shot at seeing the experiment variation.

⚠ Example Problem:

- A user lands on your page
- But the JS experiment script fails to load or they bounce before it run.
- They are counted as a visitor, but never randomized
  → leads to skewed samples.

Math Behind it:

Let define

- U: Universe of users
- SCU: Users who actually receive treatment (JS loaded + stayed long enough)
- Treatment effect:

$$\hat{T}_{observed} = \bar{E}[Y \mid Treatment, s] - \bar{E}[Y \mid control, s]$$

But what we want:

$$T_{true} = E[Y \mid treatment, U] - E[Y \mid$$

② Implications for Experiments.

when you run experiments on the client-side (browser, via Javascript), it creates new risk and challenges. them compromise:

- Randomization
- Measurement Accuracy.
- Interpretation of result.

Core Implications:

1. Exposure Bias

what it means: Not every user gets a fair shot at seeing the experiment variation.

⚠ Example Problem:

- A user lands on your page
- But the JS experiment script fails to load or they bounce before it run.
- They are counted as a visitor, but never randomized → leads to skewed samples.

Math Behind it:

let define

- U : Universe of users
- SCU : users who actually receive treatment (JS loaded + stayed long enough)
- Treatment effect:

$$\tau_{observed} = E[Y \mid Treatment, S] - E[Y \mid control, S]$$

But what we want:

$$\tau_{true} = E[Y \mid Treatment, U] - E[Y \mid control, U]$$

1. You're measuring a biased subset (S), not the whole population (U) → Classic selection bias.

2. Delayed Randomization and Logging Gaps.

"Late Registration in a Race"

- Server-side: Runners register before the race starts — you know the full list.
- Client-side: Some runners start running but don't register at the booth — you only log who crosses a timed gate, not who started.

Your final report shows only part of reality → biased conclusions.

3) Instrumentation Gaps & Noise

- JS-based.
  - ↳ Ad blockers.
  - ↳ Browser inconsistencies.
  - ↳ Network delays.
  - ↳ JS errors.

These result in:

- ↳ Missing events.
- ↳ Incorrect timestamps
- ↳ Duplicate or dropped logs.

4) Inconsistent Assignments (cross-device or cross-session).

Problem:

- ↳ Server-side assigns users based on user ID or cookie easily.
- ↳ Client-side may assign per browser session.

If the cookie is lost or JS fails → same user may get assigned different variants → variant contamination.

5. Treatment Leakage

Client-side logic might:

- load both variants initially
- Hide/show based on condition.

This causes leakage:

- users see part of treatment and control, or flickers between them.
- Might affect behaviors even before assignment → confounded metrics.

dhawa dongchising

(3) Conclusion.

(1) Client-side experiments are more fragile than they seem.

- They're easy to set up, but can easily go wrong.
- Why? Because JS-based logic depends on browser compatibility, load order, timing, network, and user behavior.
- Logging and randomization aren't as reliable.

Core takeaway: Ease of use ≠ reliability.

(2) They introduce bias - unless rigorously validated.

- Users may not be exposed correctly (due to early exits, JS issues).
- This results in inaccurate estimation of treatment effects.

Your experiment may report success, but it's really biased sampling.

(3) Server-side experiments are preferable when feasible

- They avoid:
  - Exposure bias.
  - Instrumentation failures.
  - Logging gaps.
- They're more trustworthy for critical business decision (like pricing, search algorithm changes)

Rule of thumb: If the stakes are high, go server-side.

4. When client-side is required, mitigate known issues

Examples:

- Use triggered analysis to reduce bias (unit 13)
- Ensure fast JS loading.
- Use logging Event beats or redundant logging systems.
- Validate assignment rates and exposure timing.

Run sanity check: Are assignment proportions and metric consonance normal?

(5) Underlying Math Concept: selection Bias, Missing Data and causal Inference.

In formal causal inference terms.

- server-side → satisfies SUTVA and unconfoundness (good for ATE estimation).
- Client → often violates both → leads to biased estimate of treatment effects

$$\hat{T} = \Sigma [Y_i(1) - Y_i(0)]$$

(biased if exposure is conditional)

(6) Related Concepts and Units:

Concepts	Links
Triggered Analysis (unit 13)	→ Helps restrict only to users who had a fair chance to see treatment.
Ty Twyman's Law (unit 3)	→ "If it looks too good to be true, it probably is" -

client side is vulnerable.

Observational Bias (width) → client-side exposes to random → makes the experiment observational

Missing Data (width/logging) → Js failure → missing events → violates assumptions about complete data.

(7) Summary: what should you do?

If you ...	Then ...
want to test visual or layout	→ client-side is OK, but be cautious
Are testing algo/backend logic	→ Prefer server-side for trustworthiness.
Must use client-side	→ Validate exposure, monitor assignment split.
See suspicious uplift	→ Investigate exposure? logging? bots?

Final thought:

"client-side experiment are seductive in ease, but fragile in trust." If your experiment drives product or business decisions, don't skip validation — or better yet, go server.